

Stack Tecnológico del Pipeline ETL

Asignatura: ETL

Estudiante: Wilson Andrés Gómez Gutiérrez

Antecedentes internacionales

La arquitectura de esta solución está construida bajo un enfoque de código abierto (*open-source*), priorizando la escalabilidad, la reproducibilidad y la eficiencia en el manejo de datos tabulares. El stack seleccionado se compone de las siguientes tecnologías:

1. Lenguaje de Programación Core

- **Python (3.9+)**: Seleccionado como el lenguaje orquestador por su ecosistema robusto y su estándar indiscutible en la ciencia de datos. Permite una transición fluida entre la ingeniería de datos (ETL) y la posterior fase de modelado predictivo.

2. Librerías de Extracción y Transformación (Data Wrangling)

- **Pandas**: Motor principal para la manipulación y análisis de datos estructurados. Se utiliza para la limpieza vectorial, anonimización de datos sensibles y filtrado geográfico de las bases del SIVIGILA.
- **Openpyxl**: Motor de lectura y escritura especializado que permite la ingesta directa de los archivos crudos en formato .xlsx sin necesidad de transformaciones manuales previas, automatizando el flujo desde la fuente original.
- **NumPy**: Soporte subyacente de Pandas, utilizado para el manejo eficiente de matrices y el tratamiento lógico de valores nulos (NaN) dentro de los bloques clínicos de cada evento epidemiológico.

3. Librerías Estándar y Orquestación

- **os & glob (Built-in)**: Módulos nativos de Python empleados para la interacción con el sistema operativo y el escaneo dinámico de directorios. Habilitan la capacidad del pipeline para iterar automáticamente sobre múltiples archivos (los más de 100 eventos proyectados) sin intervención manual.

4. Gestión de Entorno y Configuración

- **venv**: Herramienta nativa para el aislamiento del entorno virtual, asegurando que las dependencias del proyecto no generen conflictos con otras librerías del sistema.

- **python-dotenv:** Implementado para la futura gestión de variables de entorno, rutas absolutas y credenciales, manteniendo las buenas prácticas de seguridad en el código fuente.

5. Infraestructura y Entorno de Ejecución

- **Procesamiento Local (*Lightweight*):** En esta fase de desarrollo, el pipeline está diseñado para optimizar el consumo de memoria RAM. Su eficiencia permite procesar las matrices de datos en ráfaga sin saturar los recursos de hardware en equipos portátiles de uso personal (como la arquitectura de un HP Pavilion x360 convertible), garantizando agilidad en las pruebas locales.

Proyección de Escalabilidad (Fase Analítica): Este stack fundamental preparará los artefactos de datos (archivos limpios en formato CSV/Parquet) que servirán como insumo directo para el acoplamiento futuro con el DANE y el entrenamiento de algoritmos de Machine Learning (como Scikit-learn, XGBoost o arquitecturas de series temporales) orientados a la detección temprana y automatizada de anomalías en salud pública en el Tolima.